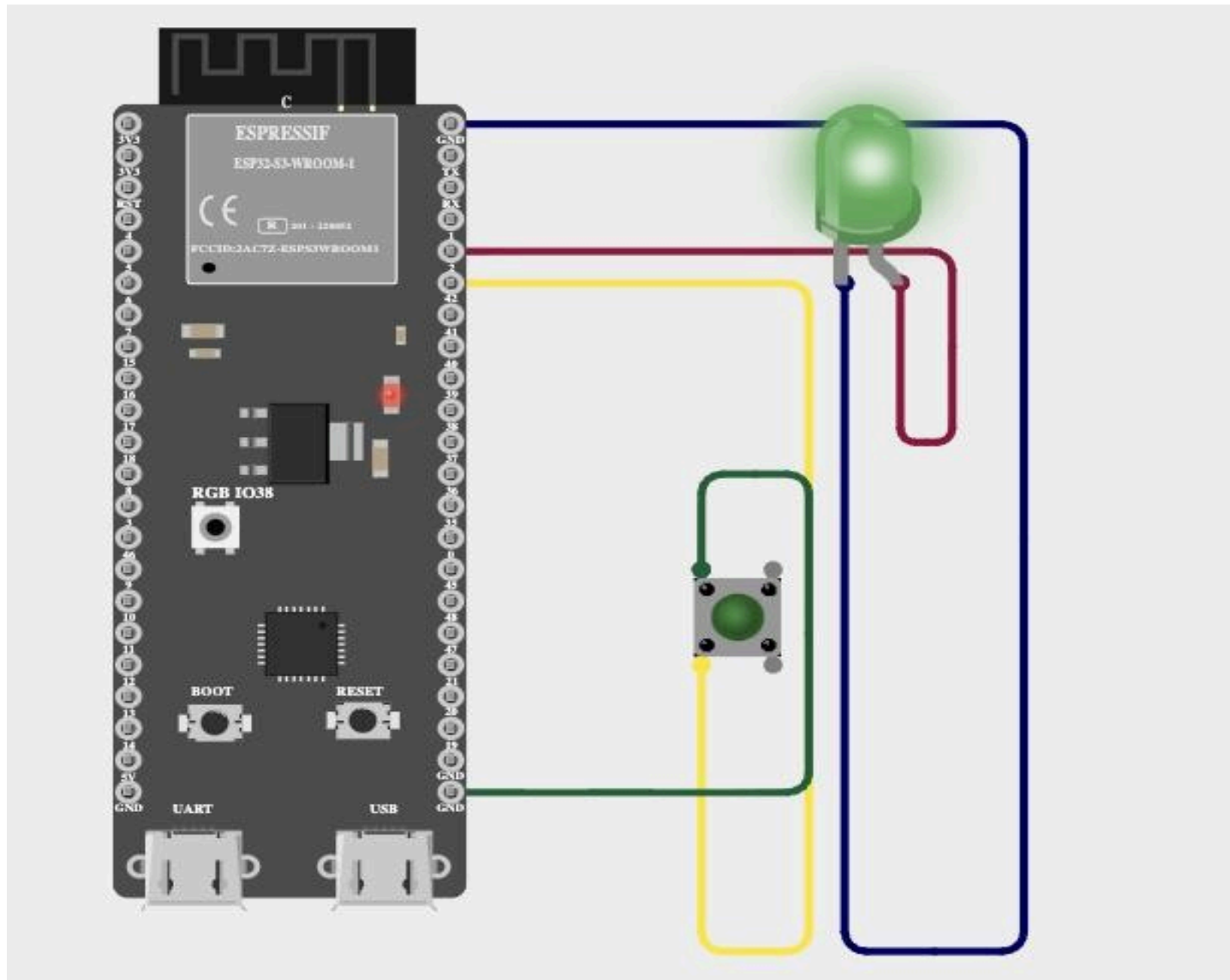


# Embedded Computing Principles

Name: Satyam shankar tripathi

## EXPERIMENT

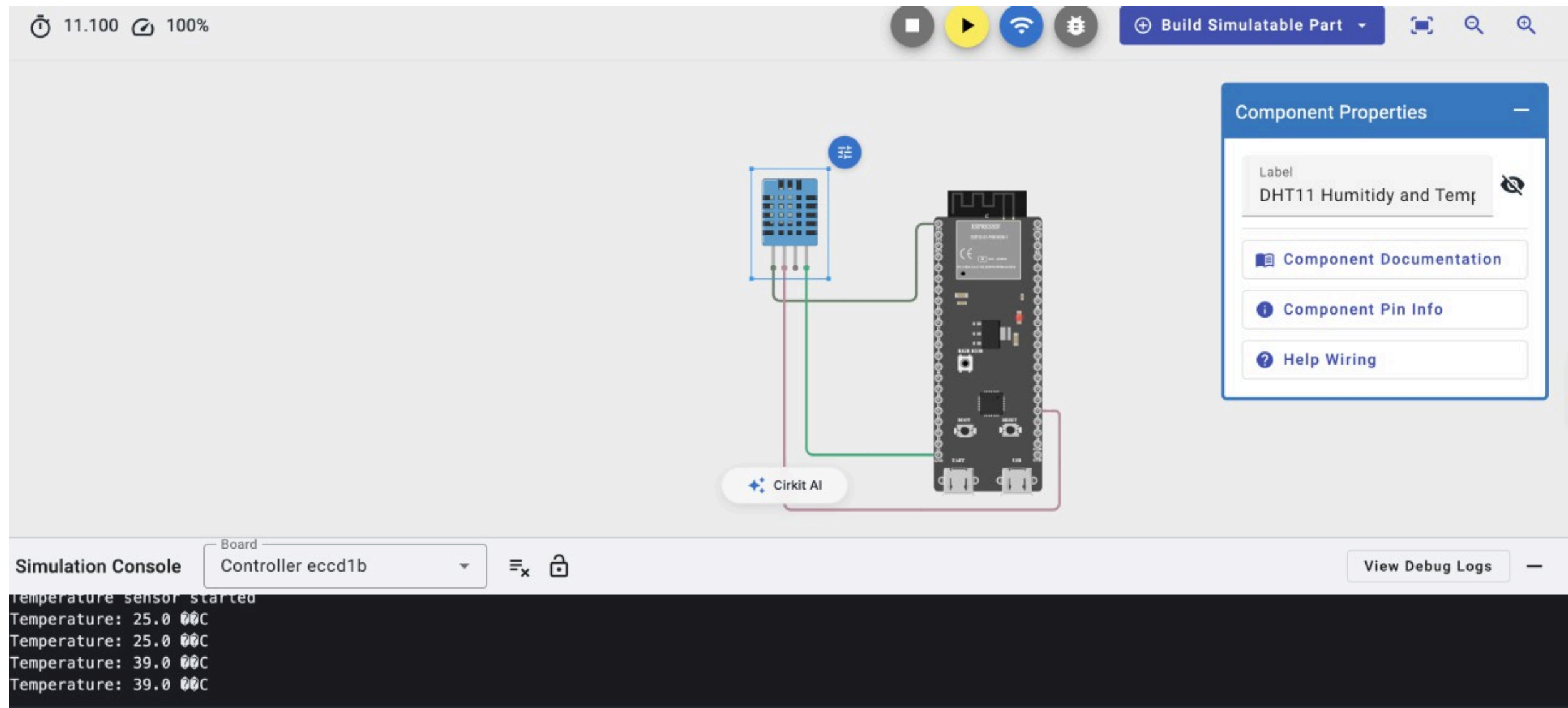
### LED Blinking s GPIO Control



### CODE:

```
const int LED_PIN = 2; void
setup() {
pinMode(LED_PIN, OUTPUT);
}
void loop() {
digitalWrite(LED_PIN, HIGH);
delay(500);
digitalWrite(LED_PIN, LOW);
delay(500);
}
```

# Temperature Monitoring



## CODE:

```
#include <DHT.h>

const uint8_t TEMPERATURE_PIN = 21;

const uint8_t SENSOR_TYPE = DHT11;

DHT temperatureSensor(TEMPERATURE_PIN, SENSOR_TYPE);

void setup() {

  Serial.begin(115200);

  delay(500); temperatureSensor.

  begin();

  Serial.println("Temperature sensor started");

}

void loop() {

  //DHT11 needs a short interval between readings. delay(2000);

  float temperatureC = temperatureSensor.readTemperature(); if

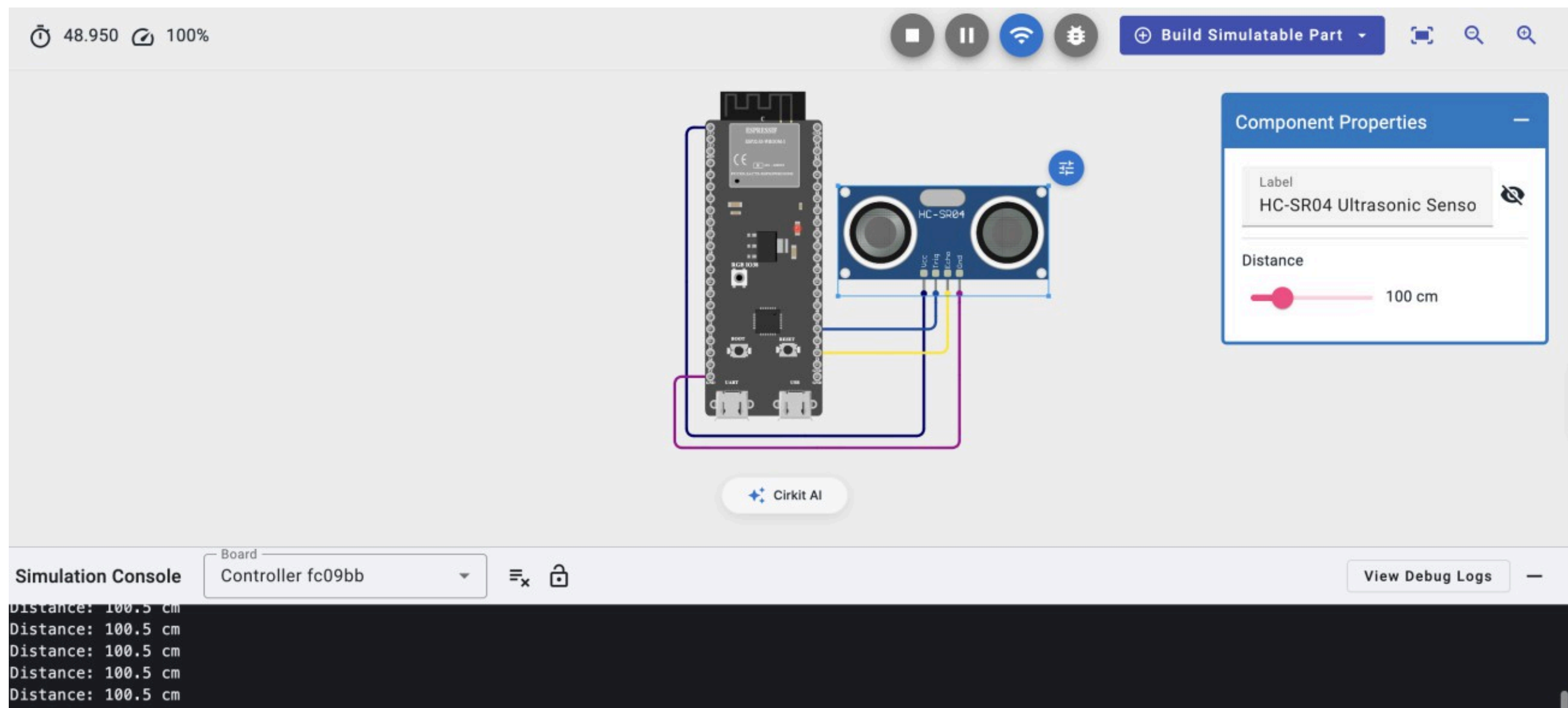
  (isnan(temperatureC)) {
```

```
Serial.println("Error: temperature reading failed");
```



```
return;  
}  
Serial.print("Temperature:  ");  
Serial.print(temperatureC, 1);  
Serial.println(" ° C");  
}
```

# Ultrasonic Distance Measurement



## CODE:

```
const int TRIG_PIN = 21;

const int ECHO_PIN = 19; void

setup() { Serial.begin(115200);

pinMode(TRIG_PIN,OUTPUT);

pinMode(ECHO_PIN,INPUT);

digitalWrite(TRIG_PIN,LOW);

delay(500);

Serial.println("HC-SR04 distance monitor started");

}

void loop(){

//Send a 10 microsecond trigger pulse.

digitalWrite(TRIG_PIN,LOW);

delayMicroseconds(2);
```

```
digitalWrite(TRIG_PIN, HIGH);

delayMicroseconds(10);

digitalWrite(TRIG_PIN, LOW);

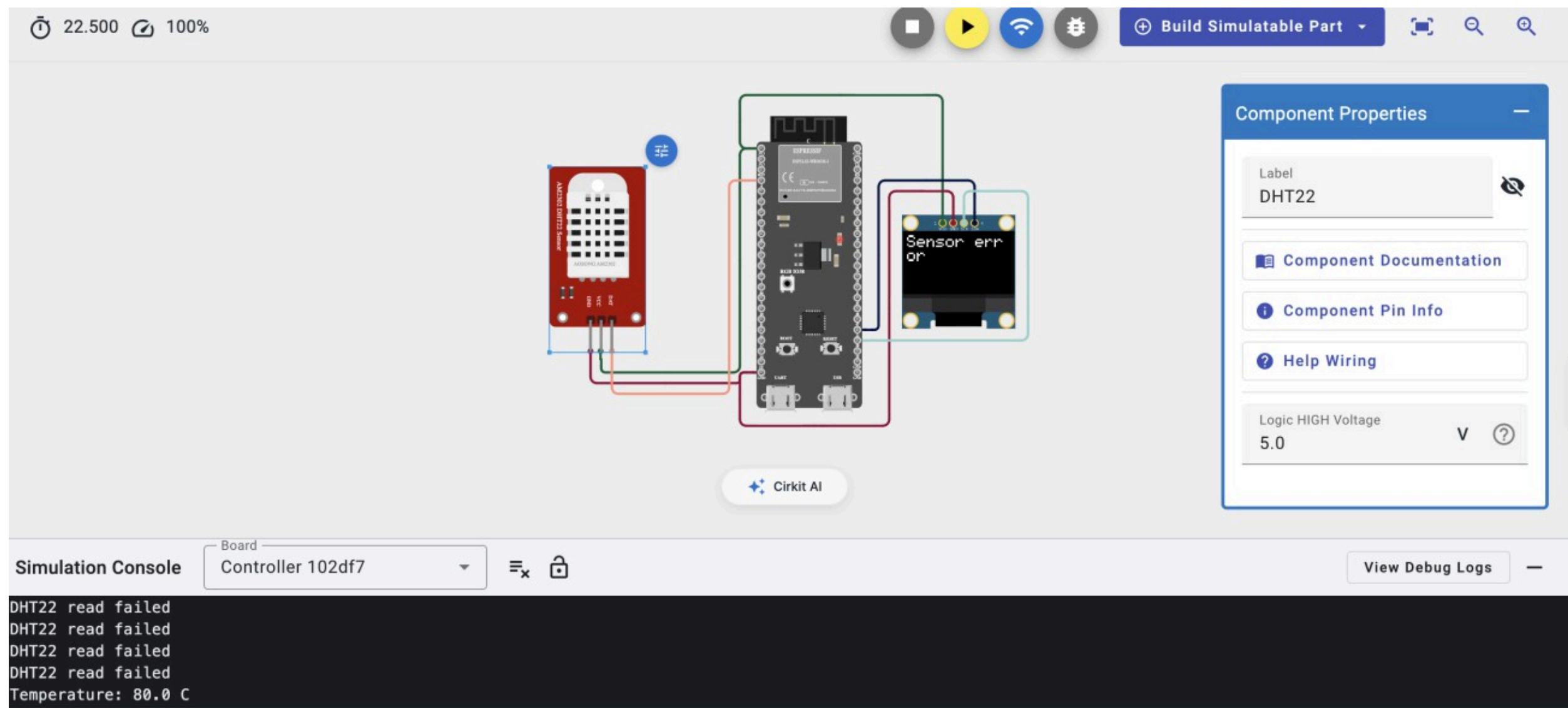
// Measure the echo time; timeout prevents the loop from hanging. unsigned
long duration = pulseIn(ECHO_PIN, HIGH, 30000);

if (duration == 0) {
  Serial.println("Distance: out of range or no echo");
} else {
  float distanceCm = duration / 58.0; Serial.
  print("Distance: ");
  Serial.print(distanceCm, 1);
  Serial.println(" cm");
}

delay(500);
}
```



## OLED Display with I<sup>2</sup>C



```
#include <Wire.h> #include
```

```
<DHT.h> #include
```

```
<Adafruit_GFX.h>
```

```
#include <Adafruit_SSD1306.h>
```

```
#define DHT_PIN 4
```

```
#define DHT_TYPE DHT22
```

```
#define SCREEN_WIDTH 128
```

```
#define SCREEN_HEIGHT 64
```

```
#define OLED_RESET -1
```

```
#define OLED_ADDRESS 0x3C
```

```
DHT dht(DHT_PIN, DHT_TYPE);
```

```
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, CWire, OLED_RESET);
```

```
void setup() {
```

```
Serial.begin(115200);
```

```
dht.begin();
```

```
Wire.begin(21, 20); // SDA, SCL
```



```
if (!display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDRESS)) {  
  Serial.println("OLED initialization failed");  
  
  while (true) {  
    delay(1000);  
  }  
}  
  
display.clearDisplay();  
  
display.setTextColor(SSD1306_WHITE);  
  
display.setTextSize(1); display.setCursor(0,  
0); display.println("Temperature monitor");  
  
display.display();  
  
delay(1000);  
}  
  
void loop() {  
  float temperatureC = dht.readTemperature(); if  
(isnan(temperatureC)) { Serial.println("DHT22  
read failed"); display.clearDisplay();  
  
  display.setCursor(0,0);  
  
  display.println("Sensor error");  
  
  display.display();  
  
} else {  
  Serial.print("Temperature: ");  
  
  Serial.print(temperatureC, 1);  
  
  Serial.println(" C");  
  
  display.clearDisplay();
```

```
display.setTextSize(1);  
  
display.setCursor(0,0);  
  
display.println("Temperature");  
  
display.setTextSize(2);  
  
display.setCursor(0,22);  
  
display.print(temperatureC,1);  
  
display.print(" C");  
  
display.display();  
  
}  
  
delay(2000); // DHT22 requires at least 2 seconds between readings  
  
}
```